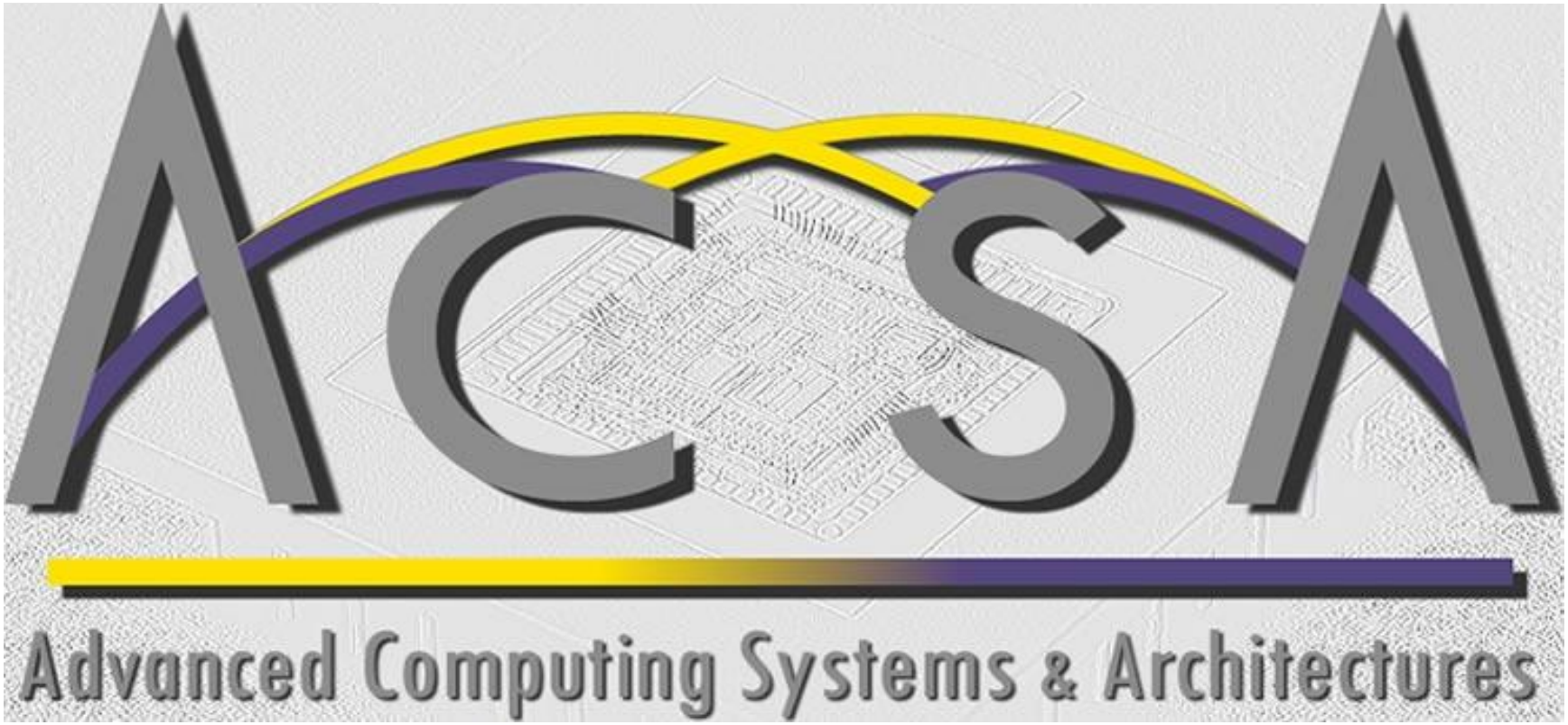


www.acsa.upt.ro

Hardware/Software Co-Design

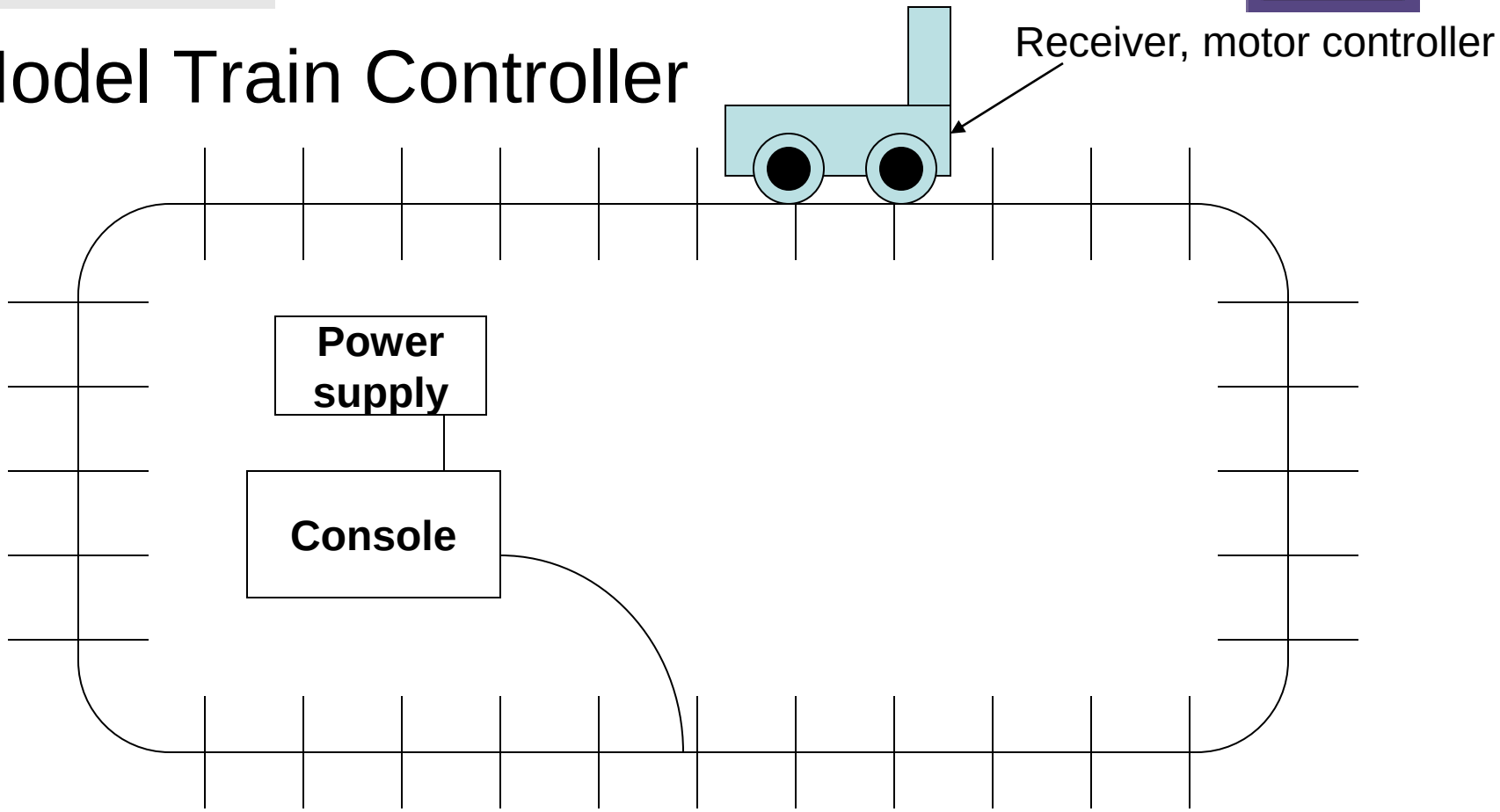
Lecture 2

1.Introduction

- Model Train Controller
  - The user sends messages to the train with a control box attached to the tracks
  - Controls: throttle, emergency stop button, etc.
  - The train receives electrical power from the tracks
    - The control box can send signals to the train over the tracks by modulating the power supply voltage
  - The control panel sends packets over the tracks to the receiver on the train
  - The train includes analog electronics for sensing the bits transmitted and a control system to set the train motor's speed and direction accordingly
  - Each packet contains an address -> the console can control several trains on the same track
    - Each packet includes an ECC part
    - The communication system is ONE-WAY

1.Introduction

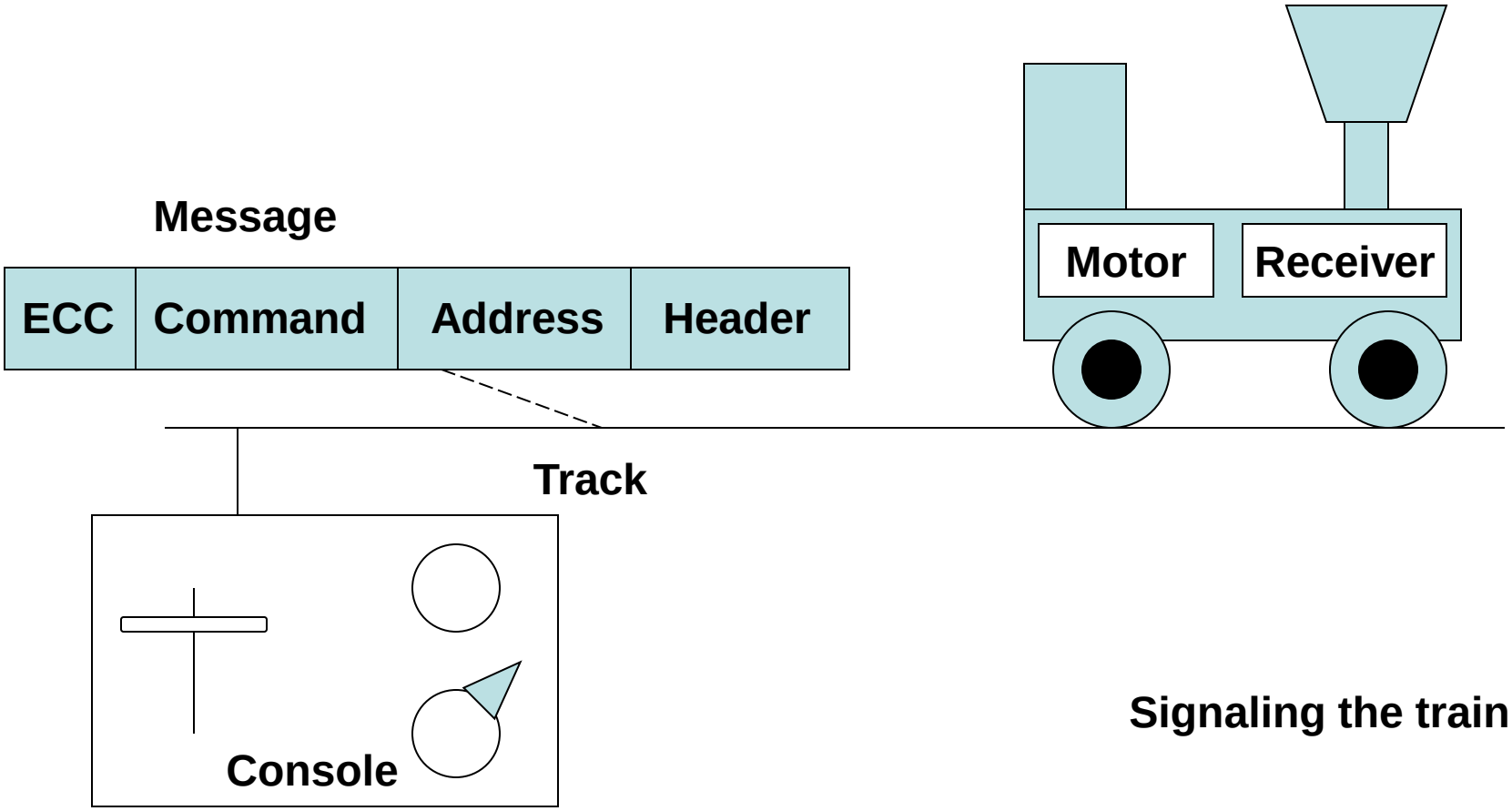
- Model Train Controller



System setup

1.Introduction

- Model Train Controller



1.Introduction

- Model Train Controller
  - Requirements (basic set description)
    - The console shall be able to control <= 8 trains on 1 track
    - The speed of each train shall be controllable by a throttle to at least 63 different levels in each directions (forward and reverse)
    - There shall be an inertia control that shall allow the user to adjust the responsiveness of the train to commanded changes in speed
    - The inertia control will provide at least 8 different levels
    - There shall be an emergency stop button
    - An error detection scheme will be used to transmit messages

1.Introduction

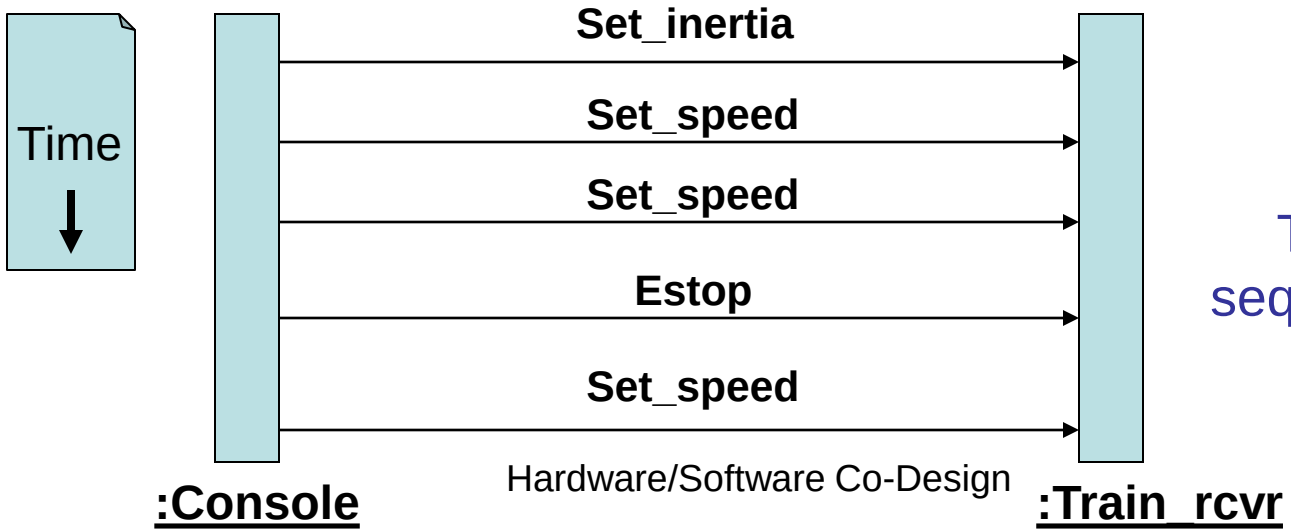
- Model Train Controller
  - Requirements

Name	**Model train controller**
Purpose	**Control speed of up to 8 model trains**
Inputs	**Throttle, inertia setting, emerging stop, train no.**
Outputs	**Train control signals**
Functions	**Set engine speed based upon inertia settings; respond to emergency stop**
Performance	**Can update train speed at least 10 times/sec.**
Manufacturing cost	**$50**
Power	**10 W (plugs into wall)**
Physical size and weight	**Console should be comfortable for 2 hands, approximate size of a standard keyboard, weight less than 2 pounds**

1.Introduction

- Model Train Controller
  - Conceptual Specification
    - The information conveyed by the messages

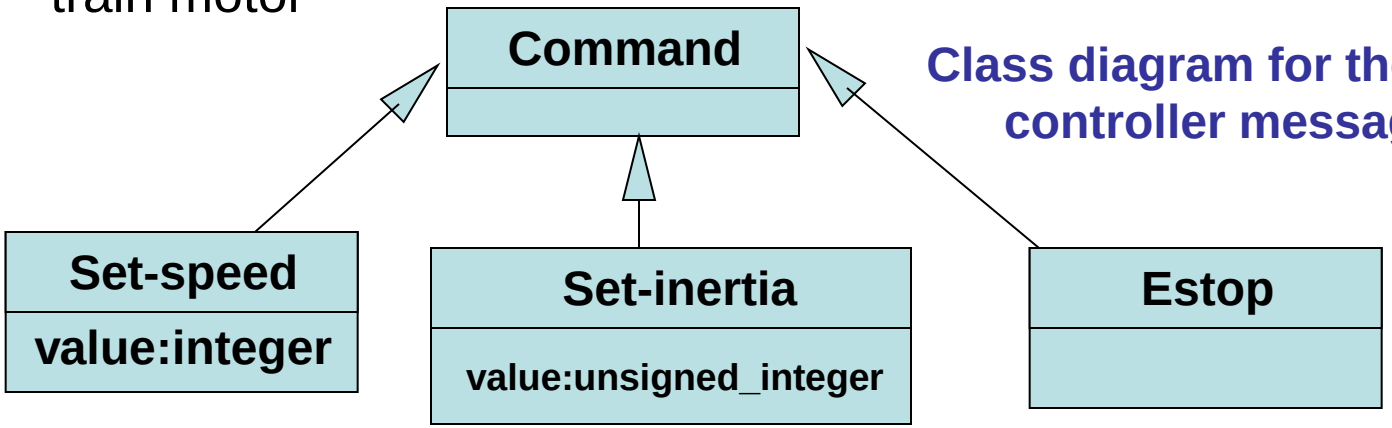
Command name	Parameters
Set-speed	**Speed (positive or negative)**
Set-inertia	**Inertia-value (nonnegative)**
Emergency stop	**None**



Typical UML sequence diagram

1.Introduction

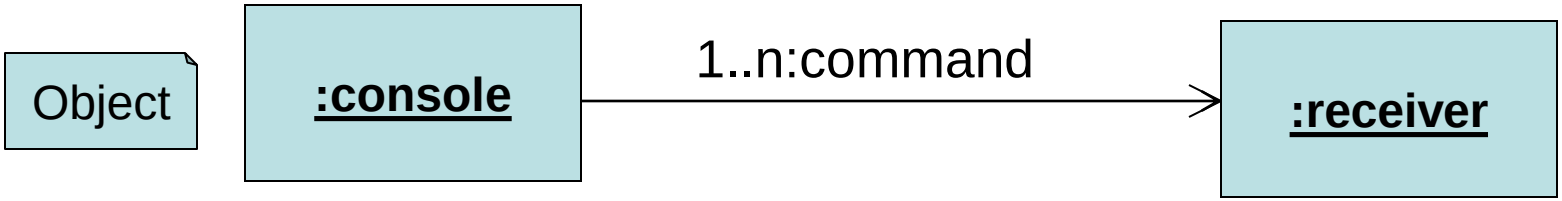
- Model Train Controller
  - Conceptual Specification
    - The information conveyed by the messages
      - When the train receives a speed command, it will speed up or slow down at a rate determined by the current inertia setting
      - Occasionally, an emergency stop may be received, which causes the train receiver to immediately shut down the train motor



Class diagram for the train controller messages

1.Introduction

- Model Train Controller
  - Conceptual Specification
    - 2 major subsystems: the control box and the train board component
    - Each of these subsystems has its own internal structure
    - **UML collaboration diagram** for major subsystems

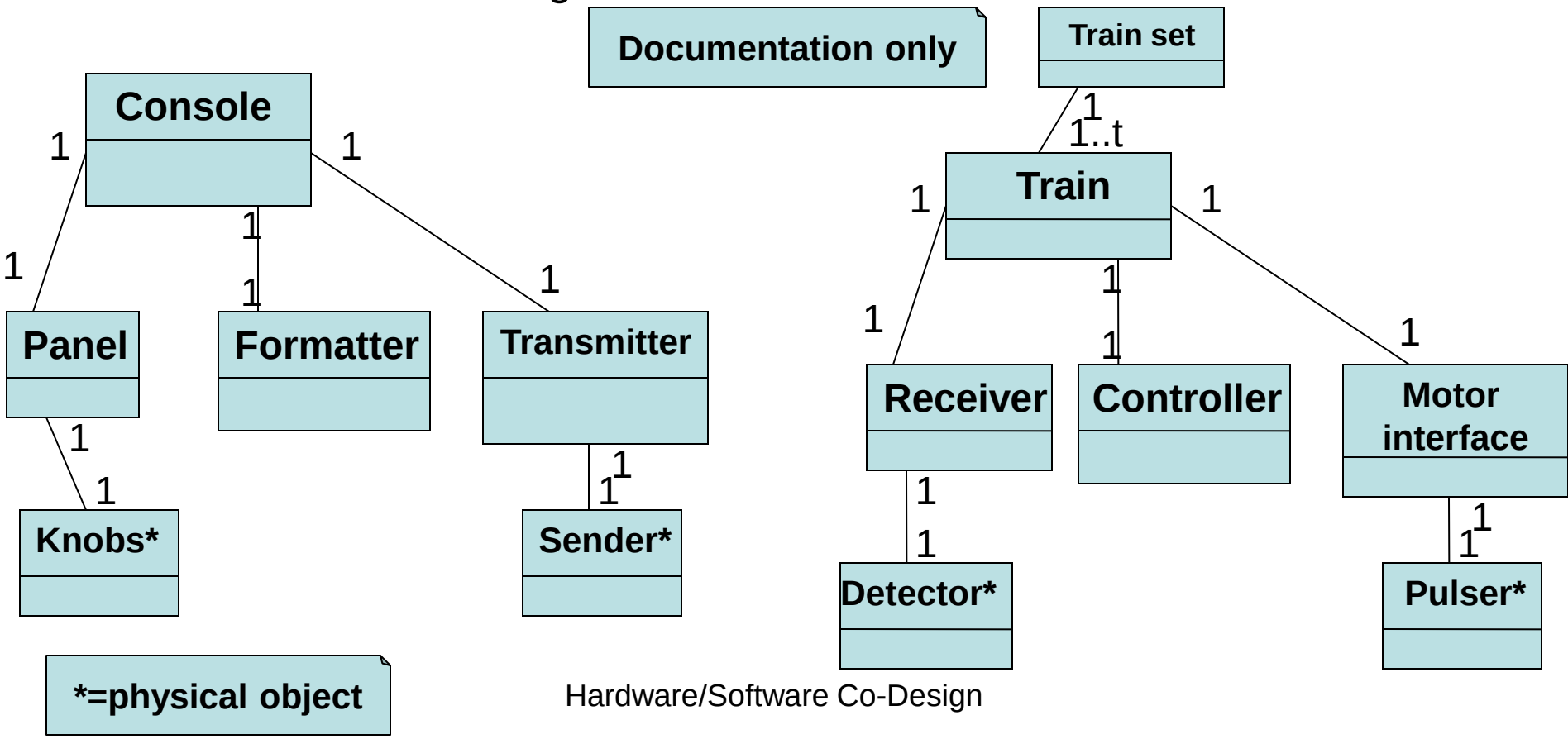


1.Introduction

- Model Train Controller
  - Conceptual Specification
    - Break down the console and receiver into their major components
      - The console needs to perform 3 functions:
        - » Read the state of the front panel
        - » Format messages
        - » Transmit messages
      - The train receiver must also perform 3 major functions
        - » Receive the message
        - » Interpret the message
        - » Actually control the motor

1.Introduction

- Model Train Controller
  - Conceptual Specification
    - UML Class diagrams: console and train receiver



1.Introduction

- Model Train Controller
  - Conceptual Specification (CONSOLE class)
    - The PANEL class: describes the console's front panel, containing analog knobs and hardware to interface to the digital part of the system
    - The FORMATTER class includes behaviors that know how to read the panel knobs and creates a bit stream for the required message
    - The TRANSMITTER class contains analog electronics to send the message along the track
    - One instance of the CONSOLE class and 1 instance of each of the component classes
    - Knobs* describes the actual analog knobs, buttons and levers on the control panel
    - Sender* describes the analog electronics that send bits along the track

1.Introduction

- Model Train Controller
  - Conceptual Specification (TRAIN class)
    - The Receiver class knows how to turn the analog signals on the track into digital form
    - The Controller class includes behavior that interpret the commands and figures out how to control the motor
    - The Motor interface class defines how to generate the analog signals required to control the motor
    - 2 classes are defined to represent analog components
      - Detector* detects analog signals on the track and converts them in digital form
      - Pulser* turns digital commands into the analog signals required to control the motor speed

1.Introduction

- Model Train Controller
  - Detailed specification

Knobs*
train-knob:integer speed-knob:integer inertia-knob:unsigned-integer emergency-stop:boolean
set-knobs()

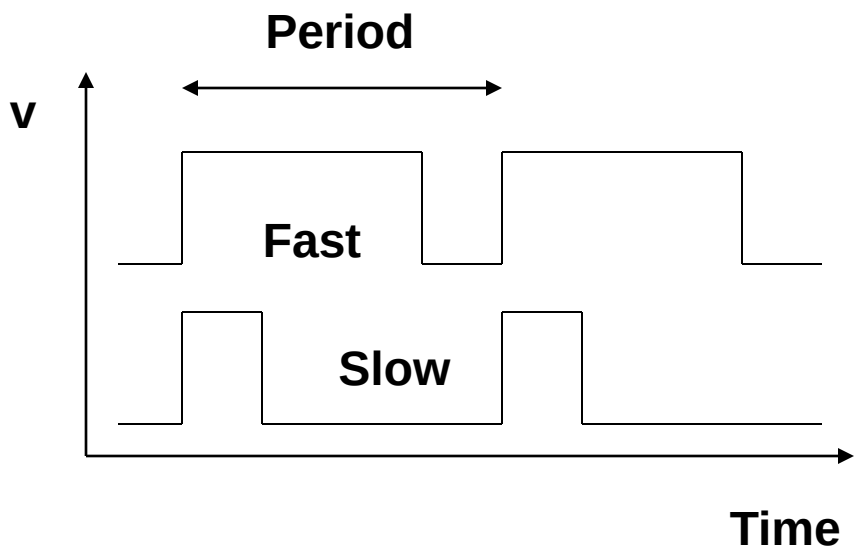
Pulser*
pulse-width:unsigned-integer direction:boolean

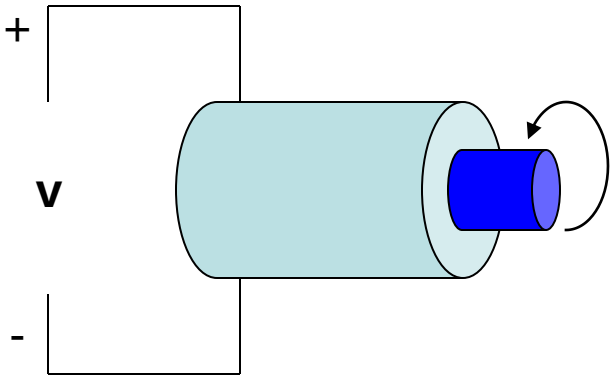
Sender*
send-bit()

Detector*
<integer>read-bit():integer

1.Introduction

- Model Train Controller
  - Detailed specification





Controlling motor speed by pulse-width modulation

1.Introduction

- Model Train Controller
  - Detailed specification (panel and motor interface)
    - These classes form the software interfaces to their respective physical devices
    - The panel class defines a behavior for each of the controls on the panel
      - **We have chosen not to define an internal variable for each control since their values can be read directly from the physical device**
      - The **new-settings** behavior uses the **set-knobs** behavior of the **Knobs*** class to change the knobs settings when the train no. is changed
      - The **motor-interface** defines an attribute for speed that can be set by other classes

Panel
train-number():integer **speed():integer** **inertia():integer** **estop():boolean** **new-settings()**

Motor-interface
speed: integer

Class diagram

1.Introduction

- Model Train Controller
  - Detailed specification (Transmitter and receiver classes)
    - Software interface to the physical devices that send and receive bits along the track
    - The transmitter provides a distinct behavior for each type of message that can be sent
      - It internally takes care of formatting the message
    - The receiver class provides a read-cmd behavior to read a message off the tracks

Transmitter
send-speed(adrs: integer, speed:integer) **send-inertia(adrs:integer, val:integer)** **send-estop(adrs:integer)**

Receiver
current: command **new: boolean**
read-cmd() **new-cmd():boolean** **rcv-type(msg-type:command)** **rcv-speed(val:integer)** **rcv-inertia(val:integer)**

Class diagram

1.Introduction

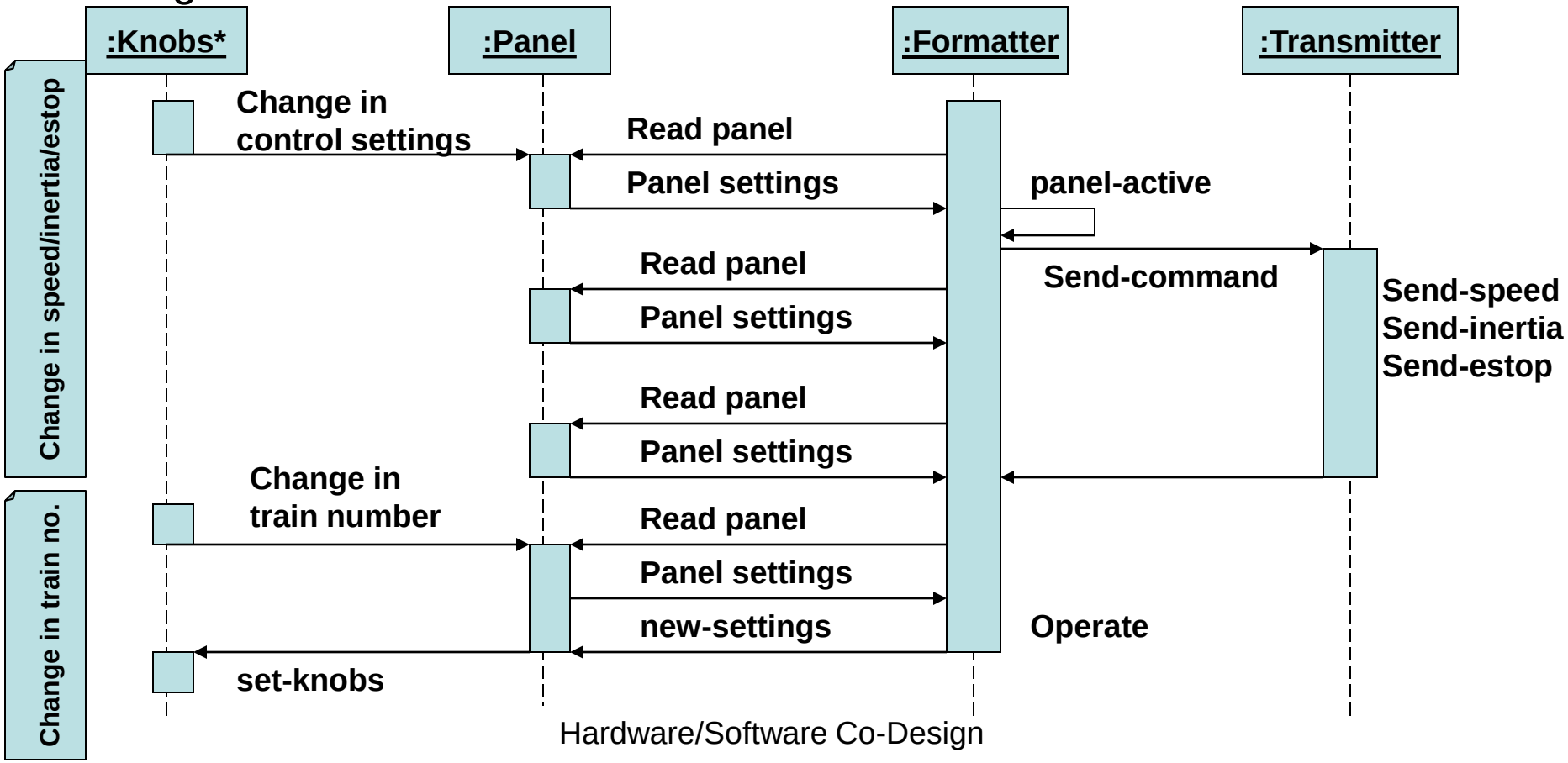
- Model Train Controller
  - Detailed specification (Formatter class)
    - Holds the current control settings for all trains
    - The send-command method is a utility function (interface to the transmitter)
    - The operate function performs the basic actions for the object
    - The panel-active behavior returns true whenever the panel‘s values do not correspond to the current values

Formatter
current-train: integer **current-speed[ntrains]: integer** **current-inertia[ntrains]: unsigned-integer** **current-estop[ntrains]:boolean**
send-command() **panel-active():boolean** **operate()**

Class diagram

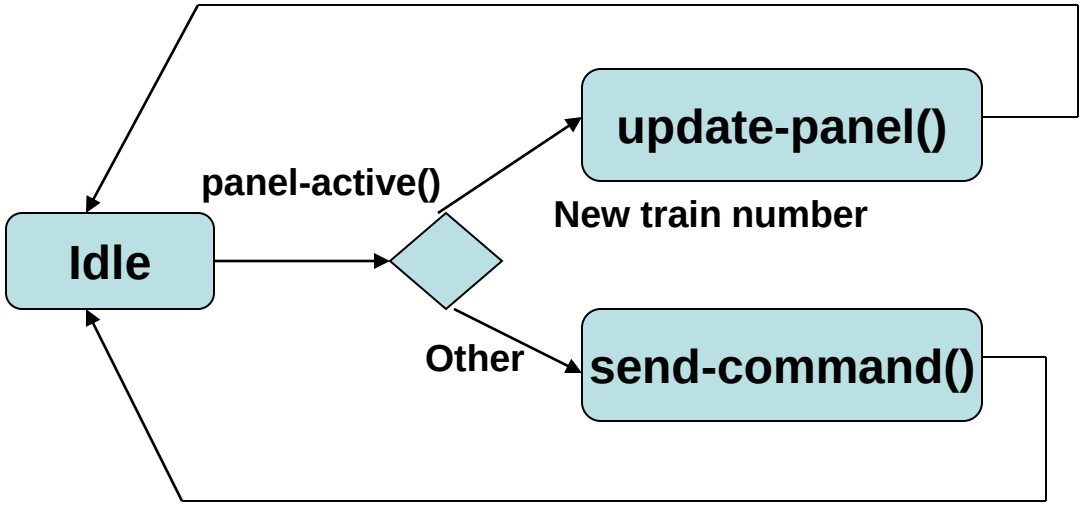
1.Introduction

- Model Train Controller
  - Detailed specification (2 Changes to the knob settings) -> Sequence diagram



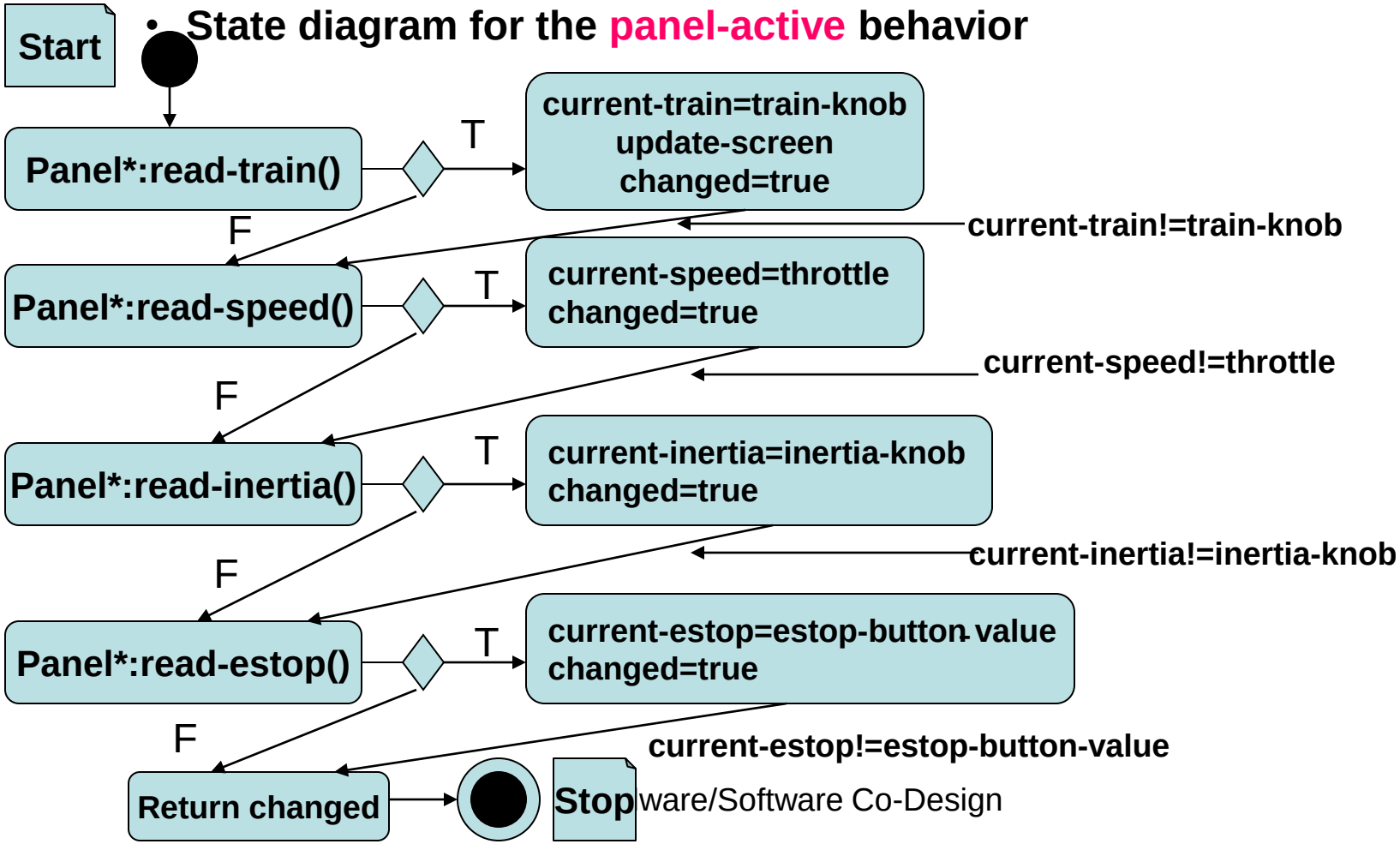
1.Introduction

- Model Train Controller
  - Detailed specification (2 Changes to the knob settings) -> Sequence diagram
    - We have not yet specified what a behavior does => we write a state diagram
    - **The state diagram for the formatter operate behavior**



1.Introduction

- Model Train Controller
  - Detailed specification (2 Changes to the knob settings) -> Sequence diagram
    - **State diagram for the panel-active behavior**

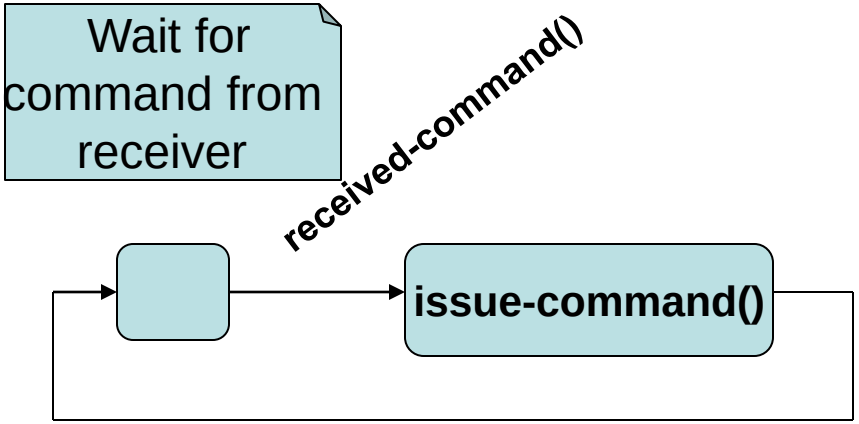


1.Introduction

- Model Train Controller
  - Detailed specification (Controller class)

Controller
current-train:integer current-speed[ntrains]:unsigned-integer current-direction[ntrains]:boolean current-inertia[ntrains]:unsigned-integer
operate() issue-command()

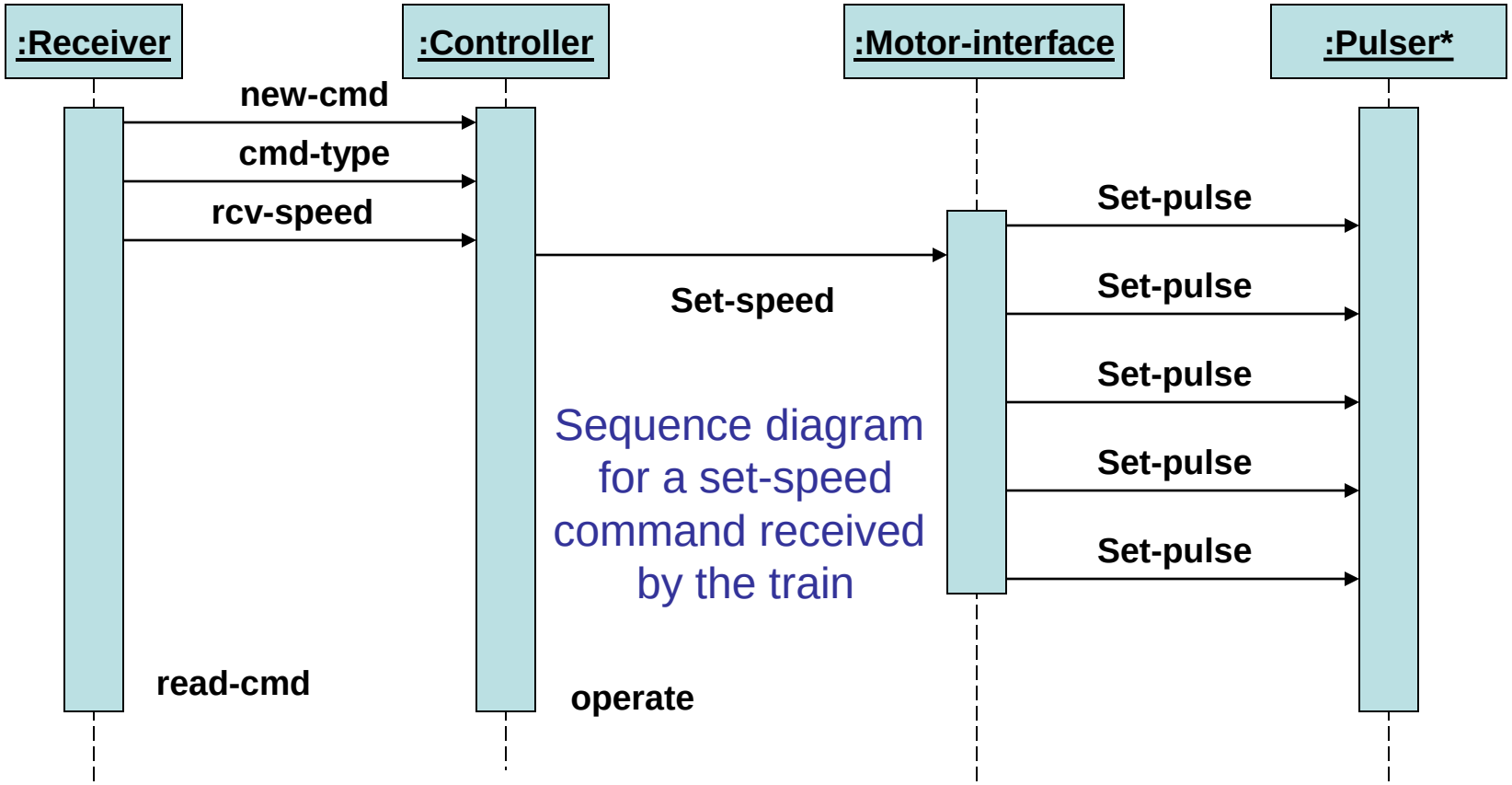
Class diagram for the Controller class



State diagram for the controller operate behavior

1.Introduction

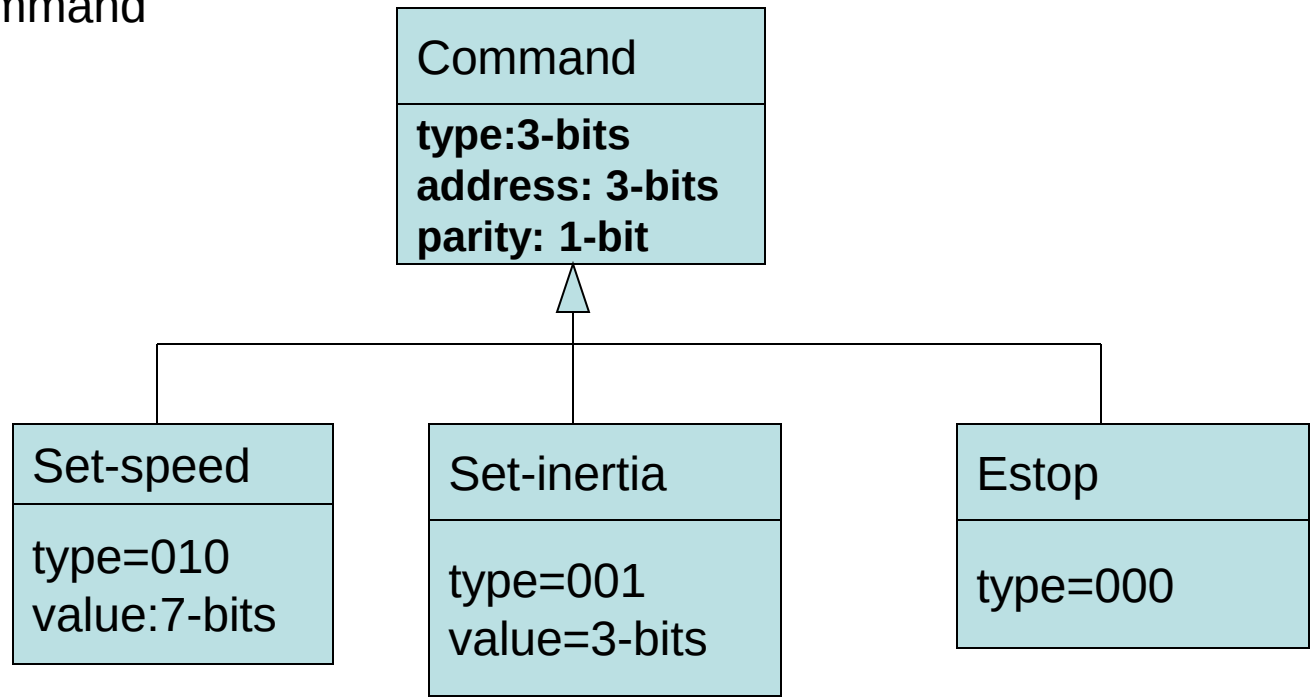
- Model Train Controller
  - Detailed specification (Controller class)
    - The operation of the Controller class during the reception of a set-speed command



Sequence diagram for a set-speed command received by the train

1.Introduction

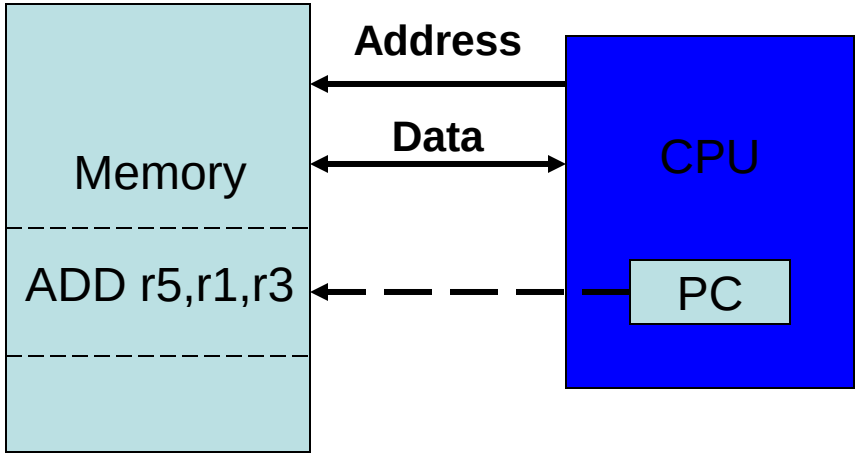
- Model Train Controller
  - Detailed specification (Controller class)
    - The operation of the Controller class during the reception of a set-speed command



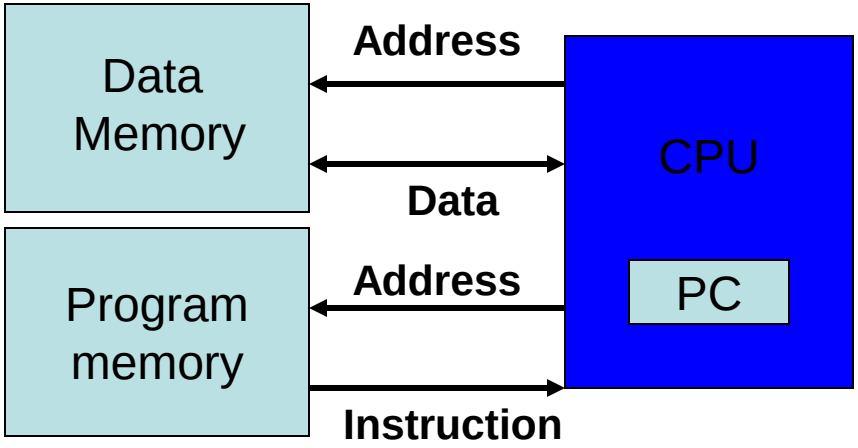
Message type = 3 bits
Data fields length -> resolution of speeds
Error control: single parity bit

2.Instruction Sets

- Instruction sets: the programmer's interface to the hardware
- The ARM processor used in PDAs (ARM version 7)
- 2.1 Preliminaries
  - Computer architecture taxonomy



A von Neumann architecture computer



A Harvard architecture

2.Instruction Sets

- 2.1 Preliminaries
  - CPU has internal registers
    - One special register is PC (holds the memory address of a instruction)
    - CPU fetches the instruction from memory, decodes it and executes it
    - Harvard architecture -> distinct data and program memory
      - Useful for DSP - processing **streaming data** (data sets arriving continuously and periodically)
      - Most DSPs today use Harvard architectures
    - Another taxonomy axis: the instruction sets
      - CISC (complex instruction set computers), many instructions
      - RISC (reduced instruction set computers), fewer instructions, pipeline techniques
      - Usually RISC are better, lately the gap has been narrowed

2.Instruction Sets

- 2.1 Preliminaries
  - Other instruction set characteristics
    - Fixed versus variable length
    - Addressing modes
    - Number of operands
    - Types of operations supported
  - Assembly Language
    - 1 instruction appears/line
    - Labels, which give names to memory locations, start in the first column
    - Instructions in the 2nd column, separate from labels
    - Comments run from some designated comment character (;)

```
Label1   ADR r4,c
         LDR r0,[r4]        ; a comment for ARM assembler
         ADR r4,d
         LDR r1,[r4]
         SUB r0,r0,r1       ; another comment
```

2.Instruction Sets

- 2.1 Preliminaries
  - Format of an ARM data processing instruction (an ADD)

31–27	26–25	24	23–20	19–15	14–11	11–0
cond	00	X	opcode	5	Rn	Rd	**Format determined by X bit**

X=1 (represents operand 2):

11–7	7–0
#rot	8-bit immediate

X=0 format:

11–6	6–4	4	3–0
#shift	Sh	0	Rm

11–7	7	6–4	4	3–0
Rs	0	Sh	1	Rm

ADDGT r0,r3,#5
cond=1100 (GT condition)
opcode=0100 (ADD)
Rn=first operand register= 3 => r3
Rd=destination register=0 => r0
operand2 is set to the immediate value of 5

2.Instruction Sets

- **2.1 Preliminaries**
  - Format of an ARM data processing instruction (an ADD)
    - Assemblers must also provide some pseudo-ops to help programmers create complete assembly language programs
    - An example of a pseudo-op is one that allows data values to be loaded into memory locations
      - These allow constants (for example) to be set into memory
      - ARM BIGBLOCK %10
      - Allocates a block of memory of the size specified by the operand and initializes those locations to 0
- 2.2 ARM Processor
  - ARM is actually a family of RISC architectures
    - Developed over many years
  - ARM does not manufacture its own VLSI devices
    - it licenses its architecture to companies who either manufacture the CPU itself or integrate the ARM processor into a larger system
  - Assembly language: textual description of the instructions as opposed to their binary representation
  - ARM instructions are written one per line, starting after the first column
  - Comments begin with a ; and continue to the end of the line

2.Instruction Sets

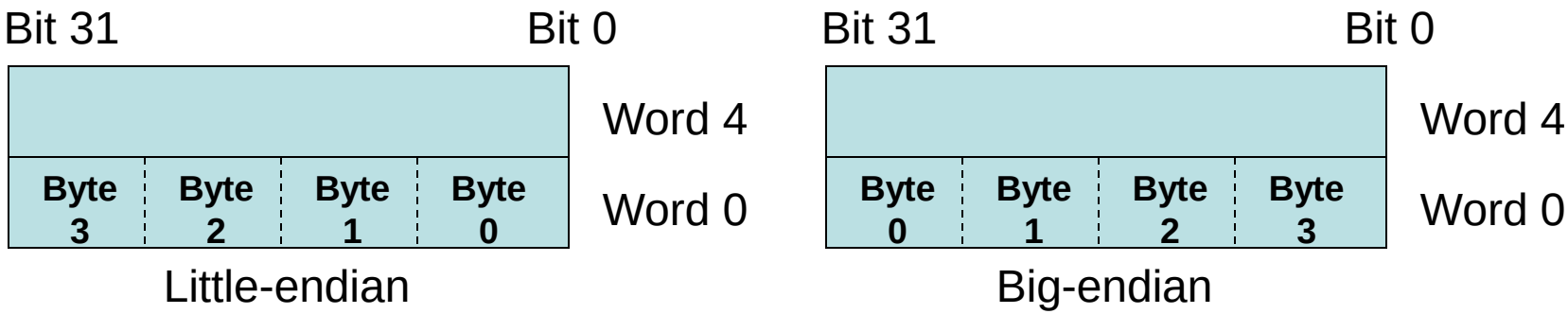
- 2.2 ARM Processor
  - Example

```
        LDR r0,[r8]; a comment
label   ADD r4,r0,r1
```

  - 2.2.1 Processor and Memory Organization
    - Different versions of ARM are identified by number
    - ARM7 is a von Neumann architecture machine
    - ARM9 uses a Harvard architecture
    - These differences are invisible to the assembly language programmer
    - The ARM architecture supports 2 basic types of data
      - The ARM standard word: 32 bits
      - The word may be divided into 4 8-bit bytes
      - ARM7 allows addresses to be 32 bits
      - An address refers to a byte, not a word (byte addresses)
        - » Word 0 is at location 0, word 1 is at location 4, and so on
        - » The PC is incremented by 4 in the absence of a branch

2.Instruction Sets

- 2.2 ARM Processor
  - 2.2.1 Processor and Memory Organization
    - The ARM processor can be configured at power-up to address the bytes in a word in either little-endian or big-endian mode



  - 2.2.2 Data Operations
    - Arithmetic and logical operations in C are performed in variables
    - Variables are implemented as memory locations
    - To write instructions to perform C expressions and assignments, we must consider both the arithmetic/logical instructions and instructions for reading and writing memory

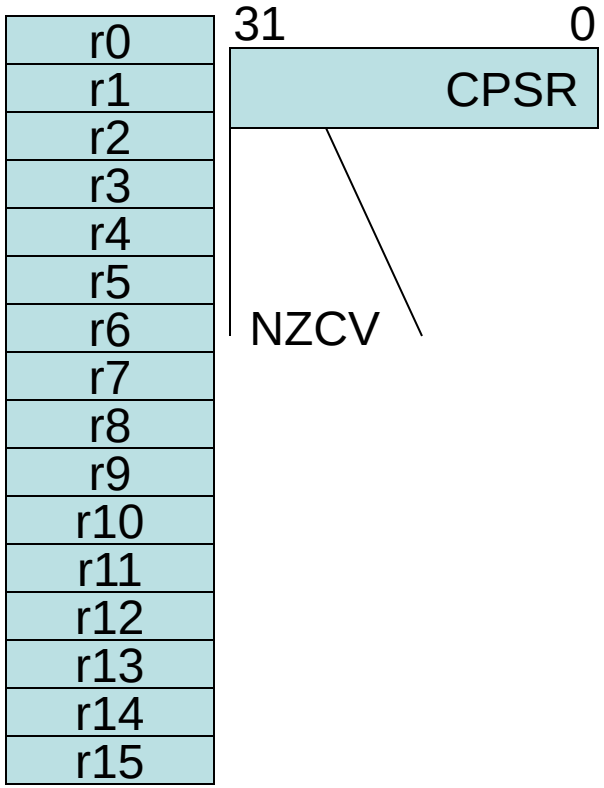
2.Instruction Sets

```
int a,b,c,x,y,z;
x=(a+b)-c;
y=a*(b+c);
z=(a<<2)|(b & 15);
```

- 2.2 ARM Processor
  - 2.2.2 Data Operations
    - A C fragment with data operations
    - The variables a,b,c,x,y,z become data locations in memory
    - In most cases **data are kept relatively separate from instructions in the program's memory image**
    - **In ARM arithmetic and logical operations cannot be performed directly on memory locations**
      - =>ARM is a **load/store** architecture
      - Data operands must first be loaded into CPU, and then the result is stored in main memory
      - The registers in the basic ARM programming model
        - » 16 general purpose registers (r0 – r15)
        - » They are identical, except r15 (which is used as PC)
        - » PC can be used as a operand in computations but should not be overwritten

2.Instruction Sets

- 2.2 ARM Processor
  - 2.2.2 Data Operations
    - The basic ARM programming model



•The other important basic register in the programming model is the **Current Program Status Register** (**CPSR**)

•CPSR is set automatically during every arithmetic, logical or shifting operation

•The top 4 bits of the CPSR hold the following useful information about the results of that arithmetic/logical operation:

- •The negative (N) bit is set when the result is negative in two's complement arithmetic
- •The zero (Z) bit is set when every bit of the result is zero
- •The carry (C) bit is set when there is a carry out of the operation
- •The overflow (V) bit is set when an arithmetic operation results in an overflow

2.Instruction Sets

- 2.2 ARM Processor
  - 2.2.2 Data Operations
    - The basic ARM programming model
      - Example for computation of the CPSR bits

An ARM word is 32 bits. In C, a hexa number starts with 0x (ex. 0xffffffff = -1 in 32-bit two's complement)

- -1+1=0 => 0xffffffff + 0x1 = 0x0, CPSR value of NZCV = 0110
- 0-1=-1 => 0x0 – 0x1= 0xffffffff, with NZCV = 1000
- 2^{31}-1+1=2^{31} => 0x7fffffff + 0x1 = 0x80000000, with NZCV=0000

- The basic form of a data instruction: ADD r0,r1,r2 ; r0=r1+r2
- Immediate operands: ADD r0,r1,#2 ; r0=r1+2

2.Instruction Sets

- 2.2 ARM Processor
  - 2.2.2 Data Operations
    - The basic ARM programming model
      - Major data operations

ADD	**Add**
ADC	**Add with carry**
SUB	**Subtract**
SBC	**Subtract with carry**
RSB	**Reverse subtract**
RSC	**Reverse subtract with carry**
MUL	**Multiply**
MLA	**Multiply and accumulate**

Arithmetic

AND	**Bitwise AND**
ORR	**Bitwise OR**
EOR	**Bitwise XOR**
BIC	**Bit clear**

Logical

LSL	**Logic shift left**
LSR	**Logical shift right**
ASL	**Arithmetic shift left**
ASR	**Arithmetic shift right**
ROR	**Rotate right**
RRX	**Rotate right extended with C**

Shift/ rotate

RSB r0,r1,r2; r0=r2-r1

BIC r0,r1,r2; r0=r1 and not r2

MLA r0, r1, r2, r3; r0=r1xr2+r3

2.Instruction Sets

- 2.2 ARM Processor
  - 2.2.2 Data Operations
    - The basic ARM programming model
      - Major data operations
        - » The shift modifier is always applied to the second source operand

CMP	**Compare**
CMN	**Negated compare**
TST	**Bitwise test**
TEQ	**Bitwise negated test**

ARM comparison instruction

MOV	**Move**
MVN	**Move negated**

ARM move instructions

LDR	**Load**
STR	**Store**
LDRH	**Load half word**
STRH	**Store half word**
LDRSH	**Load half word signed**
LDRB	**Load byte**
STRB	**Store byte**
ADR	**Set register to address**

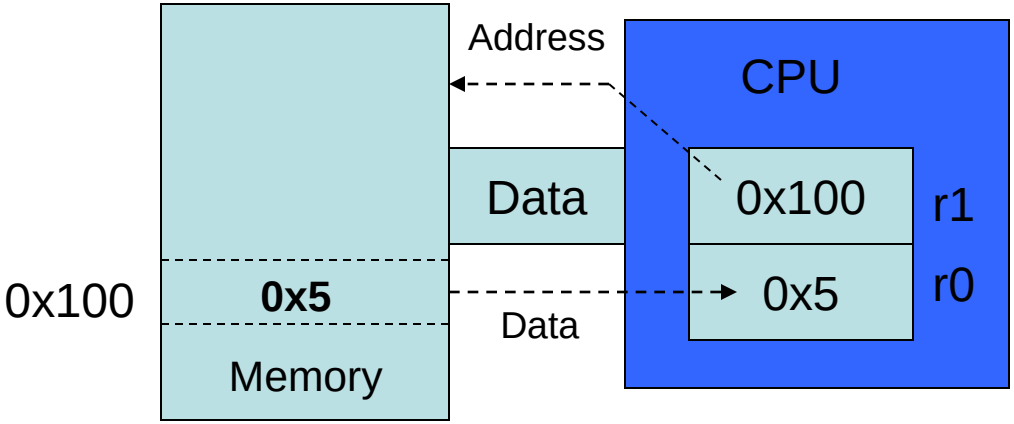
ARM load-store instructions and pseudo-operations

2.Instruction Sets

- 2.2 ARM Processor
  - 2.2.2 Data Operations
    - The basic ARM programming model
      - Major data operations
        - » The comparison instructions do not modify general purpose registers, but only NZCV bits of CPSR
        - » CMP r0,r1 computes r0-r1, sets the status bits and throws away the result of the subtraction
        - » MVN complements the operand during the move (1's complement)
        - » Load-store instructions: transfers between registers and memory
        - » The ARM load/store instructions do not directly refer to MM addresses, since a 32-bit address would not fit into an instruction that included an opcode and operands
        - » **The ARM uses register-indirect addressing**
        - » LDR r0,[r1]; sets r0 to the value of memory location 0x100 if r1=0x100
        - » STR r0,[r1]; would store the components of r0 in the memory location given in r1
        - » There are possible variations: LDR r0, [r1,- r2]

2.Instruction Sets

- 2.2 ARM Processor
  - 2.2.2 Data Operations
    - The basic ARM programming model
      - Major data operations

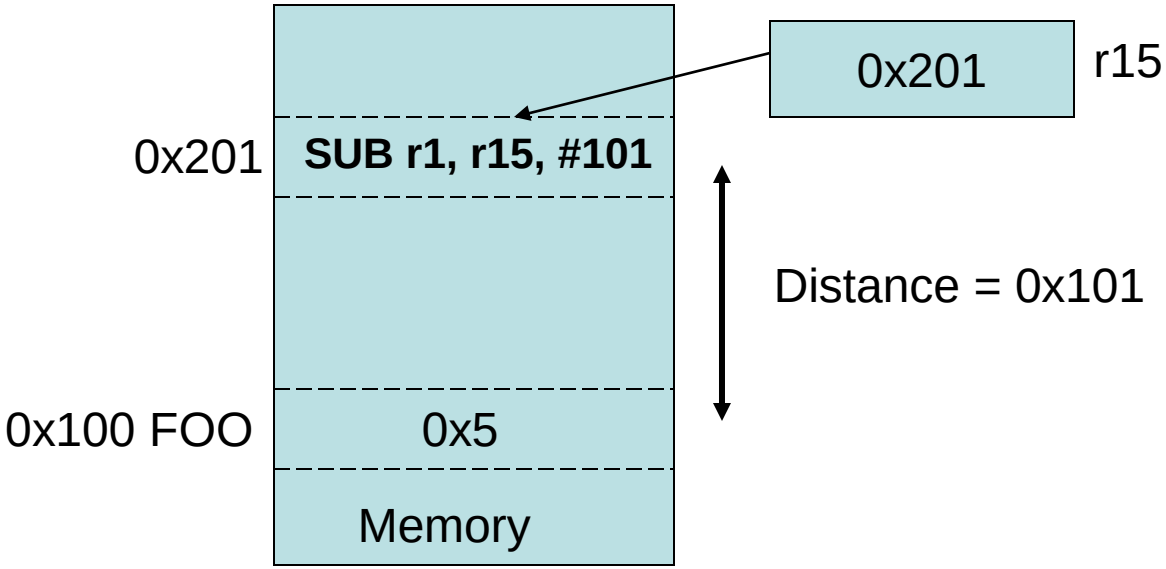


LDR r0, [r1, #4]; loads r0 from the address r1+4

LDR r0, [r1]

2.Instruction Sets

- 2.2 ARM Processor
  - 2.2.2 Data Operations
    - The basic ARM programming model
      - Major data operations



ADR r1, FOO

•We need to be able to set a register to an address (arbitrary)